

Architektura i mechanika płynnych przejść wysokości w interfejsach zakładkowych: Kompleksowa analiza paradygmatów renderowania i nowoczesnych wzorców implementacyjnych

Współczesna inżynieria interfejsów użytkownika (User Interface, UI) stanowi niezwykle skomplikowaną dziedzinę na styku projektowania wizualnego, ergonomii kognitywnej oraz niskopoziomowej optymalizacji silników renderujących. Jednym z najbardziej powszechnych, a zarazem fundamentalnie problematycznych wzorców projektowych jest implementacja systemów zakładkowych (Tabs). Służą one do organizowania złożonych struktur informacyjnych w ograniczonej przestrzeni widokowej. Oś problemu ujawnia się w momencie, gdy poszczególne zakładki zawierają treść o drastycznie różnej objętości, co w sposób naturalny wymusza zmianę fizycznych wymiarów kontenera nadrzędnego. W klasycznym, nienadzorowanym podejściu przejście między zakładkami skutkuje natychmiastowym, skokowym przesunięciem wszystkich elementów interfejsu znajdujących się poniżej aktywnego kontenera. Takie zachowanie nie tylko zaburza orientację przestrzenną użytkownika, ale jest powszechnie klasyfikowane jako krytyczny defekt wizualny rzutuujący na postrzeganą jakość całego oprogramowania.

Brak płynności w tym obszarze nie wynika z zaniedbań twórców aplikacji, lecz z fundamentalnych ograniczeń i założeń projektowych silników renderujących przeglądarek internetowych. Mechanizmy kaskadowych arkuszy stylów (CSS) przez dekady nie posiadały wbudowanej zdolności do płynnej interpolacji wartości słownikowych i wewnętrznych (intrinsic sizing keywords), takich jak `auto`, `min-content` czy `fit-content`. W związku z tym, każda próba zaaplikowania deklaratywnej tranzycji (`transition: height`) dla elementu przechodzącego z precyzyjnie określonej wartości pikselowej do wartości `auto` kończy się niepowodzeniem. Przeglądarka, nie mogąc wyliczyć w czasie rzeczywistym matematycznej ścieżki przejścia między wartością absolutną a dynamicznie kalkulowaną zawartością, ignoruje polecenie animacji, dokonując natychmiastowego skoku.

Niniejszy raport stanowi wyczerpującą, wysoce zdywersyfikowaną analizę techniczną problematyki animowania wysokości kontenerów w systemach wielostanowych. Dokument poddaje dogłębnej sekcji trzy najważniejsze braki w dotychczasowych, powszechnie stosowanych rozwiązaniach (tzw. obejściach), precyzyjnie wyjaśniając ich mechanikę oraz niszczycielski wpływ na doświadczenie użytkownika (UX) i wydajność aplikacji. W dalszej części raport proponuje i analizuje przełomowe paradygmaty architektoniczne, w tym technikę animacji FLIP, asynchroniczne wykorzystanie interfejsu `ResizeObserver`, możliwości oferowane przez Web Animations API (WAAP) oraz najnowsze specyfikacje silników przeglądarek przewidziane na lata 2024–2026, w tym rewolucyjne właściwości `interpolate-size` i funkcje matematyczne

calc-size()).

Anatomia ograniczeń technologicznych: Trzy fundamentalne braki obecnych rozwiązań

Wieloletni brak natywnych rozwiązań dla animacji dynamicznych wielkości w ekosystemie platformy Web zmusił inżynierów oprogramowania do wypracowania szeregu heurystycznych technik zastępczych. Choć techniki te pozwalały osiągnąć wizualne złudzenie płynności w ograniczonych przypadkach brzegowych, w ujęciu systemowym stanowią one przykłady długu technologicznego, obciążone krytycznymi wadami architektonicznymi. Poniżej poddano rygorystycznej analizie trzy główne paradygmaty obejść oraz ich fundamentalne braki.

Zniekształcenia krzywych czasowych w heurystyce max-height

Najstarszym, najbardziej rozpowszechnionym, a jednocześnie najbardziej obciążonym wadami mechanicznymi rozwiązaniem jest wykorzystanie właściwości max-height do symulowania animacji rozwijania kontenera. Technika ta opiera się na założeniu, że skoro przeglądarka nie potrafi wyliczyć przejścia do wartości auto, należy animować parametr, który akceptuje jednoznaczne wartości liczbowe. W tym paradygmacie element startuje z konfiguracji max-height: 0 oraz ukrytym przelewaniem treści (overflow: hidden), a po aktywacji zakładki jego wartość zmienia się na sztucznie zawyżoną, arbitralną liczbę, na przykład max-height: 1000px lub 9999px.

Fundamentalnym brakiem tego podejścia jest całkowite zniszczenie matematycznej reprezentacji krzywych czasowych (Timing Functions), co bezpośrednio przekłada się na sztuczność i brak kontroli nad dynamiką ruchu. Przeglądarka internetowa, wyliczając klatki pośrednie za pomocą funkcji takich jak ease, ease-in-out lub niestandardowych krzywych Béziera, dokonuje dystrybucji czasu proporcjonalnie do zadeklarowanego wektora odległości. Jeżeli zadeklarowany czas trwania przejścia (duration) wynosi 500 milisekund dla zmiany od 0 do 1000 pikseli, algorytm silnika optymalizuje szybkość w taki sposób, aby w wyznaczonym czasie pokonać całą tę odległość. Problem pojawia się w momencie, gdy faktyczna treść nowej zakładki zajmuje zaledwie 200 pikseli przestrzeni pionowej.

W takiej sytuacji, fizyczna zmiana wysokości widoczna na ekranie użytkownika zachodzi tylko podczas pierwszych kilkudziesięciu milisekund. Przeglądarka błyskawicznie powiększa element do fizycznych rozmiarów treści (200px), a przez pozostałą część zadeklarowanego czasu kontynuuje animację matematyczną "w tle", rozszerzając niewidzialną granicę max-height do docelowych 1000 pikseli. Prowadzi to do drastycznego i niezamierzonego przyspieszenia wizualnego samej animacji.

Jeszcze poważniejszy defekt uwiadamia się w fazie zamykania kontenera lub przełączania na zakładkę o mniejszej zawartości. Użytkownik inicjuje akcję, jednak na ekranie nie zachodzi absolutnie żadna zmiana wizualna. Dzieje się tak, ponieważ wartość max-height musi najpierw matematycznie opaść z pułapu 1000 pikseli do faktycznej wysokości 200 pikseli. Zjawisko to, w literaturze inżynierskiej określane jako "Reverse Delay" (Opóźnienie Zwrotne), tworzy w systemie martwe okna czasowe, podczas których interfejs wydaje się całkowicie zamrożony lub zepsuty. Próba mitygacji tego problemu poprzez zmniejszenie wartości max-height do poziomu nieznacznie przewyższającego spodziewaną treść niesie za sobą ryzyko "obciążenia" długich treści w przypadku urządzeń mobilnych o wąskich ekranach, na których tekst załamuje się do większej liczby wierszy. Jest to rozwiązanie charakteryzujące się skrajną kruchością

strukturalną (fragility).

Cecha mechaniczna	Prawidłowa animacja (np. docelowo height: 200px)	Wadliwa animacja (heurystyka max-height: 1000px)
Czas widocznego ruchu przy otwieraniu	Pełne 500ms (zgodnie z intencją projektanta)	~100ms (nienaturalnie przyspieszony ruch do granicy treści)
Zjawisko Reverse Delay (zamykanie)	Brak opóźnienia, natychmiastowa reakcja na bodziec	~400ms całkowitego braku wizualnej odpowiedzi interfejsu
Integracja z krzywą Béziera	Perfekcyjne wygładzenie (np. płynny start i stop)	Krzywa zdeformowana, element zatrzymuje się gwałtownie i przedwcześnie
Bezpieczeństwo wyświetlania treści	Pełna adaptacyjność do zawartości i łamania linii tekstu	Ryzyko permanentnego ucięcia zawartości (jeśli treść przekroczy limit)

Synchroniczne pomiary JavaScript i indukcja Layout Thrashing

Dostrzegając nieadekwatność i wady optyczne metody max-height, zespoły programistyczne wyewoluowały podejście polegające na dynamicznym, w locie wyliczaniu fizycznych wymiarów zawartości tuż przed zainicjowaniem przejścia. Technika ta zakłada użycie kodu JavaScript do tymczasowego wyrenderowania kontenera w trybie swobodnym (height: auto), zablokowania jego widoczności przed użytkownikiem, odczytania wyliczonych przez przeglądarkę wartości pikselowych (np. wykorzystując właściwości scrollHeight, clientHeight, offsetHeight lub precyzyjną funkcję getBoundingClientRect()), a następnie zaaplikowaniu konkretnej wartości liczbowej z powrotem do arkusza stylów. Proces ten wymusza stworzenie iluzji tranzycji między dwiema wiadomymi wartościami absolutnymi, omijając ograniczenia interpolacji słownikowej. Niestety, to rozwiązanie skrywa w sobie drugi krytyczny brak współczesnych architektur front-endowych: drastyczną ingerencję w potok renderowania (Rendering Pipeline) i indukcję zjawiska znanego jako "Layout Thrashing" (Druzgotanie Układu). Aby zrozumieć destrukcyjność tego podejścia, należy przeanalizować asynchroniczną, "leniwą" (lazy) naturę silników takich jak Blink czy WebKit. Kiedy programista zmienia strukturę DOM – na przykład poprzez usunięcie zawartości starej zakładki i wstrzyknięcie nowej – przeglądarka nie przelicza natychmiast pozycji każdego elementu na stronie. Zamiast tego flaguje poszczególne węzły drzewa renderowania jako zdezaktualizowane (dirty) i odkłada skomplikowane operacje matematyczne (Layout/Reflow) do momentu, gdy nadejdzie czas wygenerowania nowej klatki obrazu na ekran (zazwyczaj co 16.6 milisekund).

Jednakże, gdy w tej samej funkcji, bezpośrednio po wstrzyknięciu treści, kod JavaScript wymusza odczyt rozmiaru (scrollHeight), przeglądarka zostaje zmuszona do przerwania swoich optymalizacji i synchronicznego, natychmiastowego wyliczenia pozycji każdego piksela w całym dokumencie w celu zwrócenia poprawnego wyniku pomiaru do zmiennej środowiskowej. Jest to operacja wyjątkowo kosztowna obliczeniowo. Jeżeli w skomplikowanym interfejsie akcje odczytu (read) i zapisu modyfikacji stylów (write) są przeplatane wielokrotnie w trakcie trwania jednego cyklu przetwarzania, powoduje to zjawisko wymuszonego synchronicznego układania (Forced Synchronous Layout). W rezultacie, czas potrzebny na przygotowanie klatki znacząco przekracza budżet wydajnościowy, a cała aplikacja traci płynność. Animacje wysokości zbudowane na tym fundamencie, choć pozbawione "Reverse Delay", działają w sposób szarpany (jank), gubiąc klatki i w skrajnych przypadkach prowadząc do krótkotrwałego

zamrożenia wątku głównego (Main Thread).

Asynchronia cyklu renderowania i kosztowne przeliczenia kaskadowe

Trzecim zasadniczym brakiem klasycznych koncepcji animowania wymiarów kontenerów jest niezrozumienie podziału obowiązków między procesami przeglądarki, a precyzyjniej: brak wykorzystania sprzętowej izolacji warstw (Compositing). Animacyjna modyfikacja jakiejkolwiek właściwości należącej do modelu ramkowego (Box Model) – do których zalicza się nieszczęsna właściwość height, ale także width, margin czy padding – powoduje reakcję łańcuchową. Gdy kontener zwiększa swoją wysokość chociażby o jeden piksel w kolejnej klatce animacji, silnik przeglądarki musi dokonać geometrycznej rekalkulacji wszystkich elementów podrzędnych wewnątrz tego kontenera, wszystkich elementów sąsiadujących obok, a często także dokumentów nadrzędnych, które mogą zmieniać swoje własne położenie (np. w przypadku layoutów elastycznych typu Flexbox czy CSS Grid). Każda ramka animacji wymusza pełny cykl przeliczeniowy: wykonanie logiki JavaScript → Przeliczenie Layoutu (Reflow) → Nakładanie kolorów i wektorów (Paint) → Łączenie warstw (Composite). Z matematycznego punktu widzenia, wykonanie kilkudziesięciu operacji Layout i Paint na sekundę w rozbudowanych aplikacjach biznesowych jest obciążeniem niewspółmiernym do zamierzonego efektu wizualnego. Animacje modyfikujące height są skazane na wyłączną egzekucję w niezwykle zatłoczonym wątku głównym (Main Thread). Jeśli w tle zachodzi jakiegokolwiek inny proces aplikacji – na przykład deserializacja danych JSON pobranych z sieci, garbage collection silnika V8, lub renderowanie złożonego wykresu – główny wątek ulega przeciążeniu, co nieuchronnie skutkuje degradacją jakości wizualnej przejścia. Współczesne paradygmaty inżynierii nakazują unikania takich animacji i delegowanie ruchu do optymalnego, wspieranego przez procesory graficzne (GPU) wątku kompozytora, z czym klasyczne animowanie parametrów wertykalnych było całkowicie niekompatybilne.

Ergonomia kognitywna i profesjonalizm: Wpływ na User Experience (UX)

Konsekwencje wynikające z zaimplementowania wadliwych technik modyfikacji wysokości kontenerów w systemach zakładkowych niosą za sobą daleko idące, destrukcyjne reperkusje bezpośrednio w warstwie odbioru oprogramowania przez użytkownika końcowego. Interfejsy cyfrowe w dużym stopniu polegają na metaforach przestrzennych nawiązujących do świata fizycznego. Kiedy użytkownik przełącza się między dokumentami umieszczonymi na cyfrowych zakładkach, oczekuje reakcji adekwatnej do znanych mu wzorców naturalnych – płynności, przewidywalności i zasady zachowania energii.

Kognitywistyka dowodzi, iż ludzki narząd wzroku jest ewolucyjnie zaprogramowany do ciągłego śledzenia obiektów w ruchu, przy jednoczesnym generowaniu mikroprzerw percepcyjnych w trakcie błyskawicznych przesunięć gałki ocznej (sakkady). Kiedy w interfejsie następuje skokowa modyfikacja wielkości, której nie towarzyszy logiczne, czasowe rozłożenie zmiany w przestrzeni, użytkownik doznaje mikrodezorientacji. Wzrok traci punkt fiksacji, a uwaga musi zostać ponownie, świadomie zogniskowana na elementach leżących poniżej, które nagle zmieniły swoje współrzędne ekranowe. Zjawisko to, kumulując się w trakcie interakcji z aplikacją, generuje znaczny, choć początkowo podświadomy, ładunek poznawczy (Cognitive Load). Interfejs, w którym elementy "teleportują się" pod wpływem kliknięcia, jest odbierany jako szorstki, awaryjny i chaotyczny.

Ponadto, problem animacji wprost przekłada się na metrykę postrzeganej wydajności (Perceived Performance). Odpowiednio zaprojektowana animacja przejścia nie stanowi jedynie ozdobnika estetycznego; pełni funkcję psychologicznego bufora maskującego czas przetwarzania danych. W zjawisku znanym jako ślepotą na zmiany (change blindness), ludzie są skłonni ignorować opóźnienia w ładowaniu treści, o ile ich uwaga jest prowadzona płynną kinetyką. W momencie gdy wadliwe mechanizmy JavaScript wywołują zjawisko Layout Thrashing, a animacja gubi klatki, maskowanie ulega zniszczeniu, eksponując opóźnienia sprzętowe. System z deficytami płynności traci aurę profesjonalizmu, rzutując negatywnie na zaufanie do samej marki aplikacji.

W kontekście wskaźników obiektywnych stosowanych m.in. przez algorytmy analityczne i audytorskie wyszukiwarek (takich jak Core Web Vitals), skokowe przesuwanie interfejsu rzutuje na wskaźnik Cumulative Layout Shift (CLS). Profesjonalny interfejs wymaga deterministycznego, kontrolowanego zarządzania przestrzenią, w którym powiększanie kontenera jest symetrycznie połączone z ucieczką elementów sąsiadujących, nie naruszając wizualnej integralności w sposób niespodziewany. Zastosowanie nieprawidłowych heurystyk max-height czy manipulacji skryptowej bezpośrednio uderza w stabilność tych wskaźników.

Przełomowe paradygmaty rozwiązywania problemu: Architektura zaawansowana

Rozwiązanie tak głęboko zakorzenione w ekosystemie przeglądarek dylematu wymaga całkowitej redefinicji podejścia inżynierskiego. Konieczne jest porzucenie prób bezpośredniego animowania właściwości odpowiedzialnych za geometryczny przepływ dokumentu (document flow) na rzecz zaawansowanych wzorców koordynacji, sprzężenia asynchronicznego oraz natywnych, nowych standardów kaskadowych arkuszy stylów.

1. Model animacji kompozytowej: Technika FLIP

W poszukiwaniu idealnej harmonii między płynnością animacji a wydajnością procesora, przemysł front-endowy pod przewodnictwem inżynierów firmy Google wyekstrahował fundamentalny paradygmat o nazwie FLIP. Akronim ten wywodzi się od słów Pierwszy (First), Ostatni (Last), Odwrócenie (Invert) oraz Odtworzenie (Play). Koncepcja ta nie jest prostą funkcją, lecz głęboką strategią matematyczno-architektoniczną polegającą na przemieszczeniu wszystkich ciężkich kosztów obliczeniowych (Reflow/Layout) z pętli animacji do jednorazowego bufora pre-kalkulacyjnego, a sam ruch powierzyć potokowi sprzętowemu karty graficznej (Compositor Thread).

W realiach implementacji przejść wysokości między zakładkami o różnej zawartości, sekwencja wykonawcza FLIP modelowana jest w wysoce rygorystyczny sposób. W pierwszej fazie, określanej jako First, oprogramowanie natychmiast rejestruje bieżący stan układu tuż przed jego mutacją. Za pomocą precyzyjnych i natywnych funkcji geolokalizacji w dokumencie (jak `getBoundingClientRect`), układ pobiera pierwotne wielkości kontenera (np. początkowa wysokość 250 pikseli). Proces ten przebiega pasywnie i bezpiecznie.

Następnie, mechanika płynnie wkracza w fazę Last. Struktura drzewa DOM ulega całkowitej, natychmiastowej modyfikacji. Zawartość nowej zakładki zostaje wprowadzona, a kontener otrzymuje domyślną, swobodną regułę `height: auto`. Przeglądarka internetowa wykonuje błyskawiczne, skokowe przemeblowanie układu – element, jak zjawisko niepożądane pierwotnie, przybiera swoje docelowe rozmiary (na przykład wysokość 600 pikseli). Krytycznym

uwarunkowaniem w tym momencie jest to, aby operacja ta odbyła się w izolowanej pamięci i czasie przeliczeniowym, *zanim* silnik będzie miał szansę fizycznie narysować efekty tej zamiany na monitorze użytkownika (Paint). Po tej błyskawicznej, ukrytej zmianie wymiarów następuje ponowne wymierzenie geometrii.

Klucz innowacji leży w kolejnym etapie, mianowicie Invert. Inżynier posiada już pełen obraz wektorowy: wie skąd wyruszył obiekt (250px) i dokąd zmierza (600px). Obliczana jest rygorystyczna delta geometryczna reprezentująca różnicę w osi Y ($\Delta Y = -350px$). Mając elementy znajdujące się na swoich docelowych, rozszerzonych miejscach, system aplikuje odwrotną modyfikację geometryczną z wykorzystaniem sprzętowo przyspieszonej właściwości transform. Przestrzeń ulega deformacji – z reguły przez transform: `scaleY(...)` lub wymuszenie przesunięć obiektów poniżej za pomocą `translateY(...)` – co powoduje, że kontener wizualnie powraca do swoich początkowych, ciasnych granic (250px), tworząc niesamowitą iluzję stabilności w czasie.

Ostatnia sekwencja, Play, stanowi zwolnienie blokady deformacyjnej. Modyfikacja własności transform interpolowana jest z wymuszonego zniekształcenia z powrotem do stanu zerowego (transform: none lub `scaleY(1)`), przywracając tym samym naturalny widok docelowy. Ogromną zaletą tego procesu jest to, iż przez cały czas trwania przejścia nie są dokonywane żadne żądania Layoutu ani Painta; każdy pośredni piksel animacji kalkulowany jest na chipie graficznym, gwarantując niezachwianą jakość 60 klatek na sekundę (FPS). Wdrożenie techniki FLIP z wykorzystaniem kompresji `scaleY` na kontenerze głównym wymusza jednak zastosowanie architektonicznych przeciwwag: aby zachować proporcje wizualne czcionek i zagnieżdżonych obrazów przed destrukcyjnym rozciągnięciem (squish and stretch effect), wewnętrzne elementy nowej zakładki muszą otrzymać odwróconą proporcjonalnie korektę skalującą. Taka zaawansowana izolacja przestrzeni to symbol bezkompromisowego inżynierstwa.

2. Pętle asynchroniczne i potęga ResizeObserver API

Zastosowanie zaawansowanych wyliczeń matematycznych natrafia na poważną przeszkodę w świecie dynamicznych, wieloskładnikowych interfejsów webowych, w których wymiary poszczególnych widoków potrafią ewoluować nawet po ich wczytaniu. Typowym przypadkiem jest sytuacja, w której otwarta zakładka ładuje asynchroniczne obrazy lub pobiera dodatkowe bloki tekstu z zewnętrznego źródła API po czasie opóźnienia. W takich scenariuszach wymuszenie płynności nie dotyczy wyłącznie aktu przełączania zakładek, ale adaptacyjnego rezonowania całego kontenera.

Globalne obiekty i nasłuchiwanie w oknie przeglądarki (`window.onresize`) są tutaj drastycznie nieskuteczne, gdyż informują jedynie o zmianie gabarytów samego obszaru roboczego urządzenia, całkowicie milcząc w sprawach lokalnych deformacji węzłów strukturalnych. Rozwiązaniem przełomowym okazało się wyeksponowanie przez organizację W3C natywnego, precyzyjnego interfejsu `ResizeObserver`. Narzędzie to stanowi pasywny czujnik wymiarów (Bounding Box Sensor) delegowany bezpośrednio na pojedyncze elementy drzewa DOM. W przeciwieństwie do metod heurystycznych, nie odpytuje on o układ strony (nie powoduje Layout Thrashingu); działa on na zasadzie subskrypcji zdarzeń asynchronicznych emitowanych przez wewnętrzne mechanizmy silnika.

W architekturze animowanych zakładek, obiekt `ResizeObserver` tworzy swoisty nasłuch (callback) przyłączony do zawartości kontenera panelu. Za każdym razem, gdy treść ewoluuje pod wpływem dodania elementu, wczytania obrazu lub załamania się tekstu spowodowanego zmniejszeniem szerokości okna, instancja observera dostarcza tablicę obiektów wejściowych

(entries), zawierających gotowe, wysoce precyzyjne odczyty z rejonu contentRect bez ingerencji w koszty obliczeniowe.

Niezwyczajnie istotnym i niebezpiecznym aspektem integracji ResizeObserver do wymuszania zmian układu jest obsługa potoków powiadomień. Jeżeli proces nasłuchujący zmian w geometrii wywoła kod asynchroniczny, który w ramach swojej logiki jeszcze bardziej poszerzy dany element i zaaplikuje nową tranzycję w trakcie trwania tego samego renderu, silnik przeglądarki zostaje zapętłony w niekończącej się pętli odpytań i reakcji zwrotnych. Wówczas aktywowany jest bezpiecznik systemowy generujący krytyczny błąd konsolowy ResizeObserver loop limit exceeded. Komunikat ten nie jest zjawiskiem marginalnym, lecz potwierdzeniem brutalnego przerwania renderingu w celu ratowania responsywności sprzętu przed awarią.

Aby zaimplementować to rozwiązanie bezpiecznie i na wskroś profesjonalnie, specjaliści wdrażają odroczenie czasowe i synchronizację z częstotliwością odświeżania monitora, używając delegacji do natywnego mechanizmu window.requestAnimationFrame(). Oznacza to, że jakiegokolwiek korekty rozmiarów odczytane z czujnika ulegają zapisaniu do widoku w sposób paczkowany (batching) dopiero tuż przed stworzeniem nowej wizualnej klatki przez przeglądarkę, całkowicie rozrywając śmiertelną pętlę i wprowadzając najwyższy standard spójności strukturalnej.

Aspekt implementacji	Konwencjonalne zdarzenie window.resize	Precyzyjny ResizeObserver API
Zasięg monitorowania	Globalny (tylko viewport/okno)	Lokalny (wyznaczone węzły DOM)
Wykrywanie mutacji izolowanych (np. fetch API)	Nieudane (nie wykrywa przyrostu wewnętrznego)	Pełne wykrywanie zmian wewnętrznego modelu ramkowego
Profil wydajnościowy	Niski (wymaga ciągłych pomiarów wymuszających layout)	Bardzo wysoki (dostarcza wyliczone dane pasywnie)
Zarządzanie pętlą zdarzeń	Podatny na skokowe przycięcia (Event Flooding)	Wymaga ochrony przez zatorami z wykorzystaniem requestAnimationFrame

3. Kinetyczna precyzja i kontrola: Web Animations API (WAAP)

Chociaż kaskadowe arkusze stylów (CSS) gwarantują zoptymalizowane przetwarzanie ruchów opartych na z góry wiadomych tranzycjach deklaracyjnych, ich brakiem pozostaje wrodzona niemożliwość sterowania przebiegiem zdarzeń w trakcie lotu (on-the-fly). Operacje w świecie systemów wielostanowych rzadko bywają linearne. Co jeśli w trakcie 500-milisekundowej animacji rozwijania zakładki, impulsywny użytkownik natychmiast kliknie inną, mniejszą zakładkę? Interakcje napędzane klasycznym przepisaniem klas w CSS nie udostępniają żadnego transparentnego dojścia do odwracania, wstrzymywania i mieszania trwających cykli. Próby tworzenia obejść z użyciem skryptów i pętli ręcznego aktualizowania postępu przez requestAnimationFrame obarczały układ ogromnym długiem w wątku głównym.

Rozwiązaniem tych złożonych paradygmatów jest pełna adopcja Web Animations API (WAAP). Jest to potężny zbiór obiektowych interfejsów wbudowanych natywnie w nowoczesne przeglądarki, oferujący inżynierom złoty most między proceduralną, logiczną kontrolą kodu JavaScript, a głęboką, potężną wydajnością procesów akcelеровanych i kompozytowych wewnątrz silnika.

Wdrożenie WAAP pozwala na asynchroniczne formowanie ciągów kluczowych klatek

(keyframes) prosto ze skalkulowanych w locie wektorów zmiany (odczytanych wcześniej chociażby przez mechanikę FLIP czy z contentRect obserwatora). Wywołanie `element.animate()` tworzy stabilny byt instancyjny, którym można w pełni zarządzać z wykorzystaniem instrukcji takich jak `play()`, `pause()`, `reverse()` czy `cancel()`.

Fundamentalną innowacją WAAPi przydatną w środowiskach nieprzewidywalnych jest zdolność do obsługi zjawisk addytywnych i nadpisywania trajektorii bez niszczenia pierwotnego tempa przejścia. Jeżeli rozmiar kontenera zakładek znajduje się w procesie rozszerzania z 100 do 600 pikseli, a nagle zmiana wymusza modyfikację celu pośredniego, inżynier może uruchomić kolejną animację dodającą różnicę wysokości bez wywoływania optycznego zgrzytu lub natychmiastowego teleportowania interfejsu.

Co niezwykle ważne dla postrzeganej jakości obrazu (Perceived Performance), w przypadku animowania właściwości zgodnych z optymalizacją sprzętową (takich jak w macierzy transform czy opacity), instancje WAAPi uwalniają ciężar wątku głównego (Main Thread), przekazując odpowiedzialność za klatkaż bezkolizyjnie do warstwy Compositor Thread. Tak zbudowane animacje zachowują płynność nawet wtedy, gdy na pierwszym planie dochodzi do intensywnej deserializacji ogromnych formatów danych JSON czy renderowania zagnieżdżonych drzew wieloskładnikowych. Tym samym, biblioteki zewnętrzne zależne wyłącznie od obciążenia procesora podczas pętli `requestAnimationFrame` ustępują miejsca rozwiązaniom czysto przeglądarkowym.

4. Rewolucja natywnych standardów W3C (Perspektywa ewolucyjna 2024-2026)

Wszystkie wyżej przedstawione koncepcje technologiczne, pomimo swojej niezaprzeczalnej skuteczności i architektonicznej poprawności, pozostają skomplikowanymi rozwiązaniami zastępczymi wobec luki leżącej u podłoża specyfikacji kaskadowych arkuszy stylów. Ekosystem World Wide Web opiera się na stałej i nieuniknionej standaryzacji procesów powtarzalnych. Problem braku interpolacji wielkości słownikowych urósł do rangi jednego z największych bolączek w środowisku front-endowym.

Punkt krytyczny w tej dziedzinie nastąpił na przełomie lat 2024 i 2025, kiedy organizacja zarządzająca standardami Web (W3C), przy potężnym wsparciu inżynierów projektujących główne silniki (Chromium, Gecko, WebKit), ostatecznie zatwierdziła i sprowadziła na poziom stabilny rewolucyjne instrukcje niwelujące ten techniczny dysonans: deklaratywną funkcję bazową `interpolate-size` oraz wektorową instrukcję kalkulacyjną `calc-size()`. Adopcja tych standardów wyznacza koniec potrzeby orkiestracji matematycznej w JavaScript dla podstawowych animacji przepływu dokumentów.

Globalna deklaracja przejść wymiarowych: `interpolate-size: allow-keywords`

Wdrożenie przez środowiska wiodące zasady `interpolate-size: allow-keywords` pozwala na poinstruowanie kompilatora i układów kaskadowych o konieczności przeprowadzania estymacji i pełnego wsparcia przejść matematycznych dla wewnętrzzstrukturalnych wielkości determinowanych (tzw. intrinsic sizing keywords), które wcześniej były absolutnie pomijane – takich jak `auto`, `max-content`, `min-content` oraz `fit-content`.

Wykorzystanie tej instrukcji w sposób globalny (poprzez selektor głównego drzewa `:root`) natychmiast odblokowuje mechanikę, umożliwiając deweloperom wykorzystanie zaledwie kilku linii elementarnego i zwięzłego kodu do osiągnięcia doskonałego rozciągania kontenera w

zgodzie z algorytmami wygładzania przeglądarki. Należy podkreślić mechaniczne zalety tego podejścia – to sama przeglądarka internetowa, posiadając wewnętrzny dostęp do drzewa przed wyrenderowaniem zaktualizowanej warstwy wzrokowej, potrafi natychmiast z wyprzedzeniem przeliczyć dokąd będzie dążyć długość rozszerzonego układu. Rozwiązanie to wyklucza potrzebę manipulowania sztucznymi buforami wielkości (tak jak robiła to skompromitowana metoda max-height), opierając trajektorię od dokładnego wymiaru stanu początkowego (0) bezpośrednio do ustalonej wysokości absolutnej w tle ułożonego tekstu.

/ Rewolucyjny i zwięzły paradygmat animacji wymiaru na lata 2025+ */*

```
:root {
  interpolate-size: allow-keywords;
}
.tab-container {
  height: 0;
  overflow: clip; /* Bezpieczne wyc[span_22](start_span)[span_22](end_span)inanie
nadmiarowego obszaru bez obciążeń marginesów */
  transition: height 0.4s cubic-bezier(0.2, 0.0, 0, 1);
}
.tab-container.active {
  height: auto;
}
```

Konstrukty algorytmiczne w potoku renderowania: calc-size()

Drugim, komplementarnym i dającym nieporównywalną z niczym kontrolę narzędziem dostarczonym deweloperom jest asynchronicznie przeliczana kaskadowa funkcja matematyczna calc-size(). Narzędzie to redefiniuje pojęcie elastyczności. Służy ono nie tylko odblokowaniu interpolacji, ale otwiera bezprecedensowe pole do bezpiecznego dokonywania działań wewnątrz natywnych wartości wymiarowych. Co niezwykle istotne, samo wystąpienie w arkuszu stylów wezwania do wygenerowania wielkości przez calc-size() automatycznie narzuca na wierzchołek elementu zasady interpolate-size: allow-keywords, zdejmując z deweloperów przymus dodawania globalnych definicji z otoczenia.

Architektura tej nowatorskiej funkcji opiera się na przyjmowaniu dwóch zintegrowanych argumentów: podstawy obliczeniowej (calc-size basis) wskazującej na wiodące słowo kluczowe definiujące ułożenie dokumentu (np. fit-content lub auto) oraz bloku dokonującego obliczeń z użyciem zarezerwowanej, pre-kalkulowanej zmiennej słownikowej nazwanej precyzyjnie size.

Pozwala to nie tylko na bezpieczne animowanie rozciągnięcia kontenera zakładki bez kosztownego środowiska JavaScript, lecz chociażby na proporcjonalne przeliczanie wymiaru przestrzeni dodatkowej z nieznanego wyjściowego pola z zawartością tekstu:

/ Przejście w tryb docelowy poszerzony o bezpieczne marginesy buforowe */*

```
.tab-wrapper.expanded {
  height: calc-size(auto, size + 20px);
}
```

Należy niezwykle stanowczo pochylić się nad strategią wdrażania progresywnego (Progressive Enhancement), albowiem implementacja nowych API podlega asymetrycznemu przyswajaniu przez głównych operatorów rynkowych systemów klienckich. Jak obrazują zaawansowane metryki globalnego śledzenia środowiskowego oraz dokumentacja analityczna dla przeglądarek

z maja 2026 roku, instrukcje interpolate-size (również wraz z wariantami i wytycznymi w numeric-only) oraz domeny calc-size() operują płynnie w zunifikowanym i ustabilizowanym ekosystemie układów wiodących, obejmujących przeglądarkę Google Chrome od przełomowej wersji 129 wzwyż. Podobne stabilne i kompleksowe poparcie techniczne dotyczy linii Microsoft Edge (od środowiska 129).

Niezwyczajnie ważną różnicą na arenie wsparcia (weryfikowalną do roku 2026) są systemy operacyjne Apple oraz oprogramowanie spod egidy Mozilla. Narzędzia z domeny WebKit obsługujące środowiska deweloperskie i stabilne edycje przeglądarki Safari odrzuciły pełną zgodność dla kompilowania calc-size() w wersjach stabilnych obejmujących luki dla Safari 26.4, choć postępy analityczne nie wykluczają wprowadzania modułów na niższych warstwach eksperymentalnych (TP). Przeglądarka Mozilla Firefox również wykazuje duży sceptycyzm wprowadzania technologii w stabilnych edycjach wielowątkowych obejmujących wdrożenia od wersji 150.

Wersja oprogramowania na rynkach w 2026 r.	Wsparcie dyrektywy interpolate-size	Zgodność kompilacji funkcji calc-size()	Adopcja technologii mobilnych (Mobile Web)
Google Chrome / Blink	Pełne wsparcie, wdrożone od v129	Pełne wdrożenie analityczne od v129	Chrome for Android pełna stabilność od v147
Microsoft Edge	Stabilne i oficjalne (v129+)	Oficjalnie zaimplementowane (v129+)	Całkowita kompatybilność ze środowiskiem
Mozilla Firefox / Gecko	Zablokowane wsparcie w warstwie v150	Zablokowane wprowadzanie do wersji 150	Częściowa nieobsługiwana standaryzacja
Apple Safari / WebKit	Brak obszernego wsparcia na poziomie 26.4	Blokada i nieznane środowisko 26.4 - TP	Zignorowanie przez system środowiska iOS
Opera Browser	Obsługa bezkolizyjna od wersji 115-127	Stabilne wkomponowanie od wersji 115+	Pełne przyswojenie dla rynków niezależnych

Ze względu na to obiektywne rozwarstwienie adaptacyjne, środowisko inżynierijne bezwzględnie nie jest w stanie oprzeć stu procent odpowiedzialności korporacyjnej o nowe wytyczne W3C. Z tego powodu powoływane w życie są zaawansowane bloki ochronne obejmujące użycie zabezpieczenia środowiskowego @supports(height: calc-size(auto, size)) {...} wraz ze wspomagającymi blokami architektury w warstwie logiki.

Architektura profesjonalnych komponentów wielostanowych w nowoczesnych systemach

Budowanie wysokiej rangi systemów wielostanowych dla środowisk izolujących DOM, do których kwalifikuje się zdecydowana elita narzędzi strukturalnych takich jak React.js, Vue czy ujednoliconych Web Components, nie może opierać się o naiwne wprowadzanie jednego rozwiązania. Obowiązkiem i paradygmatem inżynierii programistycznej jest tworzenie wysoce odpornych form (Resilient Systems) – systemów asymetrycznych i hybrydowych. Zaawansowana, architektoniczna odpowiedź na opisywane dylematy sprowadza się do złożenia

modelu FLIP, koordynacji poprzez Web Animations API (WAAPi), a także dynamicznej weryfikacji wymiarów opartych o ResizeObserver API na twardym i solidnym podparciu metod upraszczających z użyciem reguł interpolate-size.

Zalecany wzorzec tworzenia wysoce zoptymalizowanego kontenera układu operującego na systemach zakładek bazuje na precyzyjnym podziale przebiegu na dyskretne, ściśle nadzorowane stany wywodzące się z modelu koncepcyjnego Finite State Machine:

1. **Faza Detekcji Środowiskowej (Environment Detection):** Kontener u samej podstawy dokonywania renderu wyjściowego po raz pierwszy (mount phase), wykorzystując globalne i bezpieczne nasłuchiwanie w kodzie, dokonuje inspekcji systemu klienckiego pod względem dostępności API. Następuje wywołanie globalnego sprawdzenia możliwości na zewnątrz algorytmów np. za pomocą definicji @supports (interpolate-size: allow-keywords) lub programowej zmiennej CSS.supports. Jeżeli środowisko znajduje się w spektrum przeglądarek z roku 2025 i udostępnia kompilację (takich jak wiodący moduł Chromium) – system w całości dezaktywuje niepotrzebny koszt kalkulacyjny wynikający z inicjalizacji ResizeObserver i blokuje przeprowadzanie zagnieżdżonego ruchu w układzie skryptowym JavaScript. Narzucany jest tryb deklaracyjny i delegacyjny spoczywający na karku zoptymalizowanego w warstwie C++ modułu układania arkuszy stylów kaskadowych.
2. **Stan Uśpiony i Gotowości (Idle State):** Kontener panelowy nie wykazuje ruchu. Posiada predefiniowany dostęp do warstw widokowych, charakteryzuje się włączeniem wycinania na styku nieprzewidzianych marginesów (overflow: clip). Zawartość utrzymywana jest w bezpiecznym trybie odcięcia na poziomie osi.
3. **Punkt Wychylenia (Triggering Phase):** Użytkownik angażuje obszar nawigacyjny, inicjując wywołanie asynchroniczne podyktowane żądaniem z widoku. System (maszyna stanu) odbiera polecenie wykonania i przed wywołaniem mutacji (re-render component), wylicza w czasie rzeczywistym bazową zmienną położenia przestrzennego, wpisując rygorystycznie uzyskany obszar pionowy pobrany z metody pomiarowej getBoundingClientRect().height do pamięci niezależnej powiązanej z życiem samej biblioteki (mechanika First w ideologii FLIP).
4. **Modyfikacja Zagnieżdżona (Structural Mutation):** Przeglądarka z potężnym i izolowanym systemem usuwa poprzedni układ z drzewa widokowego i aplikuje nową pętlę tekstową oraz pobrane asynchroniczne komponenty graficzne do zdefiniowanego rejonu. Ponieważ węzły dom są uwolnione w środowisku macierzy do struktury o zadeklarowanych marginesach swobodnych (height: auto), zajmują one natychmiast maksymalną ilość przewidzianej i naturalnej strefy. Operacja jest poddawana kontroli bez przerysowywania wizualizacji na wyjściu ekranu, zatrzymana we fragmencie przetwarzania.
5. **Dystrybucja Ruchu poprzez WAAPi (Kinetic Orchestration):** Uaktywniony zostaje element mechaniki pomiarowej przed pojawieniem się klatki renderu powiązanego ze strefą Last. Weryfikowany zostaje dystans końcowy powiązany i oznaczany na podstawie docelowych wyliczeń. Wtedy zostaje zintegrowany obiekt instancji wywoływany dzięki komendzie WAAPi element.animate(). Architektura dynamicznie tworzy paczkę precyzyjnych zmiennych przypisywanych w tablicy kluczy przemieszczania, od wymiaru First do parametru Last z nałożonym z góry czasem z systemu desygnacyjnego komponentu (Design System Tokens) nakładając specyficzne załagodzenia matematyczne przypisane na funkcję zmiennej. Moduł WAAPi przesyła instrukcje natychmiastowo na zlokalizowany po stronie karty graficznej i niezblokowany od strony głównej warstwowy obszar Compositora.

6. **Fail-Safe poprzez Asynchroniczną Korelację Obserwacji (Adaptive Healing):**

Wykraczając poza klasyczny horyzont, aby środowisko posiadało niezachwianą precyzję, wewnątrz bloku w locie wprowadzany jest pomiar pasywny dokonywany asynchronicznym wglądem dzięki instancji interfejsu powiązanego z wymiarami z użyciem modułu ResizeObserver API. Przesyłanie na bieżąco zmiennych dokonywane na styku dołączonych obrazków odbywa się po odłożeniu odpytania za pomocą natywnego bloku window.requestAnimationFrame(). Oznacza to rewelacyjną powtarzalność powiązaną z usuwaniem kolizji blokad wątkowych znanych powszechnie ze starszych systemów jako obarczający błąd przekroczenia zatoru pętli (Loop limit exceeded).

7. **Harmonizacja Zakończenia (Settled State):** Po finalnym osiągnięciu odległości wektorowej przez obiekt wykonujący proces wewnątrz silnika WAAPI uruchomiona zostaje komenda czyszcząca i przywracająca kontenerowi natywną funkcjonalność strumieniową. Pozwala to na niezakłócone wchodzenie elementu w responsywne zarządzanie zawartością z ominięciem jakiegokolwiek wymuszonej zmiennej na sztywno opartej na szarym wektorze wykreowanym w momencie tworzenia.

Tak skonstruowana, zdywersyfikowana i hybrydowa inżynieria potrafi oprzeć bezawaryjne pole widzenia pod presją tysięcy wyświetleń, przy maksymalnym obciążeniu zasobów maszynowych i skomplikowanych animacjach nakładanych synchronicznie na rozliczne warstwy front-endu. Z analizy strukturalnej płynie absolutnie jednoznaczny przekaz obnażający wieloletnie i niezwykle powielane błędy wprowadzane do systemów. Archaiczne algorytmy opierające swoje działanie o rzekomo intuicyjne ukrywanie założeń matematycznych poza zasięg wzroku, z wykorzystaniem limitów oznaczonych u góry parametrami max-height okazały się rażąco deformować zaszczerpione w potok instrukcje kinetyczne z upływającym nienaturalnym ruchem i powtarzającym się zawieszaniem przestrzeni (Reverse Delay). Destrukcyjne wymuszanie ponownych kalkulacji i obarczanie procesora po stronie użytkownika wywołane pętlami wymierzania natywnego scrollHeight doprowadziły ekosystem pod ryzyko rwania i kruszenia wydajnościowych budżetów systemowych prowadząc prostą ścieżką w objęcia zabójczego dla klatek błędu z obszaru tak zwanego Layout Thrashing.

Zastosowanie wzorów kompozytowych oddzielających strefy przeliczania z użyciem technologii matematycznej izolacji (z modelem FLIP wyznaczającym rytm obrotu wektorów), przenoszenia ciężaru z przestarzałych asynchronicznych odpytań ze wsparciem sprzężonej kontroli i stabilnego reagowania nadawanych przez asystę interfejsu potężnego z użyciem obiektów sterowanych z warstwy powłoki z ramienia Web Animations API, po rewolucję na etapie ewolucyjnej standaryzacji (roczniki 2024–2026), wyznaczonej przez natywne uwolnienie zmiennych interpolacyjnych z opcjonalnym zastosowaniem elastycznego mechanizmu na poziomie samego kompilatora ze strony instrukcji wyliczeniowej calc-size() oraz interpolate-size: allow-keywords, zwiastuje włączenie zdezaktualizowanych metod do muzeum i podniesienie norm wykonawczych z wyznaczeniem nowego standardu ergonomii sprzętowej dla nowo implementowanych środowisk z zaawansowaną przestrzenią graficzną, z szacunkiem w odniesieniu do bezwzględnych standardów kognitywistyki cyfrowej.

Cytowane prace

1. Animate to height: auto; (and other intrinsic sizing keywords) in CSS ..., <https://developer.chrome.com/docs/css-ui/animate-to-height-auto>
2. Solving Dynamic Height Animations with Native CSS Tools | by adi carmel - Medium, <https://medium.com/@carmeladi/solving-dynamic-height-animations-with-native-css-tools-c96723db4382>
3. How To Do CSS Transitions With Height: Auto - Carl Anderson,

<https://carlanderson.xyz/how-to-animate-on-height-auto/> 4. Using ResizeObserver and transitions for animating container with ..., <https://medium.com/@sheppard28/using-resizeobserver-and-transitions-for-animating-container-with-dynamic-content-d804bb69b6cb> 5. How to adapt changing tabs JS to transition height on tab change? - Stack Overflow, <https://stackoverflow.com/questions/53803913/how-to-adapt-changing-tabs-js-to-transition-height-on-tab-change> 6. ResizeObserver: it's like document.onresize for elements | Articles - web.dev, <https://web.dev/articles/resize-observer> 7. Everything You Need to Know About FLIP Animations in React - CSS-Tricks, <https://css-tricks.com/everything-you-need-to-know-about-flip-animations-in-react/> 8. The FLIP technique for list animations - Josh Comeau, <https://www.joshwcomeau.com/react/animating-the-unanimatable/> 9. Layout being triggered on css transform - Stack Overflow, <https://stackoverflow.com/questions/37497307/layout-being-triggered-on-css-transform> 10. Changing height to auto only gets applied at the end instead of during the animation - GSAP, <https://gsap.com/community/forums/topic/34263-changing-height-to-auto-only-gets-applied-at-the-end-instead-of-during-the-animation/> 11. Do you still need Framer Motion? | Motion Magazine, <https://motion.dev/magazine/do-you-still-need-framer-motion> 12. What are the Performance/Efficiency implications of setting Web Animation API's currentTime property continuously? - Stack Overflow, <https://stackoverflow.com/questions/71470298/what-are-the-performance-efficiency-implications-of-setting-web-animation-apis> 13. The Web Animation Performance Tier List | Motion Magazine, <https://motion.dev/magazine/web-animation-performance-tier-list> 14. A Comparison of Animation Technologies - CSS-Tricks, <https://css-tricks.com/comparison-animation-technologies/> 15. CSS Animation Performance Tips | theproductguy.in, <https://theproductguy.in/blogs/css-keyframes-performance/> 16. The Web Animation API - Telerik.com, <https://www.telerik.com/blogs/web-animation-api> 17. Animating Layouts with the FLIP Technique - CSS-Tricks, <https://css-tricks.com/animating-layouts-with-the-flip-technique/> 18. Using The FLIP Animation Technique For Impossible Layout Animations - YouTube, https://www.youtube.com/watch?v=idD9DA9eR_A 19. container CSS at-rule - MDN Web Docs, <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@container> 20. How to: Create a Reflection - WPF | Microsoft Learn, <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/graphics-multimedia/how-to-create-a-reflection> 21. Stacking Divs from Bottom to Top - Stack Overflow, <https://stackoverflow.com/questions/6401869/stacking-divs-from-bottom-to-top> 22. Keep Aspect Ratio on scale() - GSAP, <https://gsap.com/community/forums/topic/44660-keep-aspect-ratio-on-scale/> 23. ResizeObserver - Web APIs | MDN, <https://developer.mozilla.org/en-US/docs/Web/API/ResizeObserver> 24. Resize Observer API - MDN Web Docs - Mozilla, https://developer.mozilla.org/en-US/docs/Web/API/ResizeObserver_API 25. Resolving "Resize Observer Loop Limit Exceeded" Errors - DhiWise, <https://www.dhiwise.com/post/resolving-resize-observer-loop-limit-exceeded-errors> 26. Taming ResizeObserver errors - jonelantha: Blog, <https://blog.elantha.com/resizeobserver-loop-limit-exceeded> 27. google chrome - ResizeObserver - loop limit exceeded - Stack Overflow, <https://stackoverflow.com/questions/49384120/resizeobserver-loop-limit-exceeded> 28. How to

fix "ResizeObserver loop completed with undelivered notifications" - Stack Overflow, <https://stackoverflow.com/questions/76187282/how-to-fix-resizeobserver-loop-completed-with-undelivered-notifications> 29. ResizeObserver loop limit exceeded · Issue #1548 · atlassian/react-beautiful-dnd - GitHub, <https://github.com/atlassian/react-beautiful-dnd/issues/1548> 30. How do I use ResizeObserver with requestAnimationFrame correctly - Stack Overflow, <https://stackoverflow.com/questions/77842752/how-do-i-use-resizeobserver-with-requestanimationframe-correctly> 31. When to use WAAP | Web Animation API | Documentation, <https://animejs.com/documentation/web-animation-api/when-to-use-waapi/> 32. CSS versus JavaScript animations | Articles - web.dev, <https://web.dev/articles/css-vs-javascript> 33. Using the Web Animations API - MDN Web Docs, https://developer.mozilla.org/en-US/docs/Web/API/Web_Animations_API/Using_the_Web_Animations_API 34. Animation - Shoelace, <https://shoelace.style/components/animation> 35. How to Animate the Details Element Using WAAP - CSS-Tricks, <https://css-tricks.com/how-to-animate-the-details-element-using-waapi/> 36. Additive Animation with the Web Animations API - CSS-Tricks, <https://css-tricks.com/additive-animation-web-animations-api/> 37. override the styles set by web animation api - Stack Overflow, <https://stackoverflow.com/questions/46203660/override-the-styles-set-by-web-animation-api> 38. interpolate-size CSS property - MDN Web Docs, <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/interpolate-size> 39. calc-size() and interpolate size | 12 Days of Web, <https://12daysofweb.dev/2024/calc-size-and-interpolate-size/> 40. calc-size() - CSS-Tricks, <https://css-tricks.com/almanac/functions/c/calc-size/> 41. What You Need to Know about Modern CSS (2025 Edition) - Frontend Masters, <https://frontendmasters.com/blog/what-you-need-to-know-about-modern-css-2025-edition/> 42. 10 CSS Features That Replace JavaScript — No Libraries, No Build Step - BigDevSoon, <https://bigdevsoon.me/blog/css-replaces-javascript-2026/> 43. CSS interpolate-size to the rescue · September 25, 2024 - Adam Argyle, <https://nerdy.dev/interpolate-size> 44. calc-size() | Can I use... Support tables for HTML5, CSS3, etc - CanIUse, <https://caniuse.com/wf-calc-size> 45. "numeric" | Can I use... Support tables for HTML5, CSS3, etc - CanIUse, <https://caniuse.com/?search=numeric> 46. CSS property: interpolate-size: allow-keywords | Can I use... Support tables for HTML5, CSS3, etc - CanIUse, https://caniuse.com/mdn-css_properties_interpolate-size_allow-keywords 47. "interpolate-size" | Can I use... Support tables for HTML5, CSS3, etc - CanIUse, <https://caniuse.com/?search=interpolate-size> 48. CSS property: interpolate-size | Can I use... Support tables for HTML5, CSS3, etc - CanIUse, https://caniuse.com/mdn-css_properties_interpolate-size 49. types: `calc-size()` | Can I use... Support tables for HTML5, CSS3, etc - CanIUse, https://caniuse.com/mdn-css_types_calc-size 50. Let's Play Accordion with the HTML details element - Oida, is des org!, <https://www.oidaisdes.org/blog/lets-play-accordion/>